



AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH

Faculty of Science and Technology
Department of Computer Science
[Cours Title: Web Technologies]

Semester: [e.g. Summer 2025-26]

Group No: [__01]

Section: [__AA]

Group Members:

SL.	Student ID	Name	Role	Signature
1	24-58618-2	Safwan Mahmud	Group Leader	
2	24-58529-2	Tunajjina Sums	Member	
3	24-58613-2	Shahriar Hasan Nafis	Member	
4	24-58639-2	Fahan Hamlan Babar	Member	

Project Title

KhabarLagbe – Online Food Ordering & Delivery Management System

Short Description

KhabarLagbe is a web-based food ordering and delivery management system that connects customers, restaurants, delivery riders, and a system administrator on a single platform. Customers can browse restaurants and food items, place orders for pickup or delivery, track their orders, and rate their experience. Restaurant owners can manage their menu, validate incoming orders, and assign delivery riders. Riders can view their assigned deliveries and update delivery status, while communicating with customers directly. The Admin oversees the entire platform — managing users, monitoring revenue, acting on customer feedback, and handling payments. The system is built using PHP (MVC architecture) and MySQL, and was developed and tested locally using XAMPP.

Introduction

Ordering food online has become part of everyday life, and a reliable platform needs to coordinate several different parties at once — the customer placing the order, the restaurant preparing it, the rider delivering it, and an administrator keeping the whole platform running smoothly. Handling these roles separately, or relying on manual coordination (phone calls, paper order slips, or spreadsheets), often leads to delays, lost orders, and poor communication between the restaurant and the delivery rider.

KhabarLagbe (“Food is needed”) is developed to solve this problem by providing a centralized, role-based web application. Each type of user — Customer, Restaurant, Rider, and Admin — logs into their own dashboard with features relevant to their responsibilities. Orders flow automatically from the customer, through the restaurant's

validation and rider assignment, to final delivery, with status updates and messaging available at every stage. This makes the ordering process faster, more transparent, and easier to manage than manual alternatives.

Background Study

Traditionally, small and medium restaurants take orders over the phone or in person and coordinate deliveries informally, using riders who are contacted individually for each order. This makes it difficult to track order status, manage payments consistently, collect customer feedback, or analyze revenue and performance over time. As the number of daily orders grows, this manual approach becomes error-prone and hard to scale.

Web-based food ordering platforms address this by integrating ordering, restaurant management, delivery assignment, and payment tracking into one system, with each stakeholder given the specific tools they need. KhabarLagbe follows this model: it separates concerns cleanly using an MVC (Model-View-Controller) architecture in PHP, backed by a MySQL database, so that orders, users, restaurants, foods, payments, feedback, and delivery status are all tracked in structured tables rather than scattered records. The platform was built and tested in a local XAMPP environment (Apache + MySQL + PHP), which is a common, lightweight setup for developing and demonstrating this type of web application before deployment.

Objectives

- Provide a single web platform where customers, restaurants, riders, and admin staff can each carry out their own tasks without phone calls or paper records.
- Let customers browse multiple restaurants and place orders for either home delivery or self pickup.
- Give restaurant owners full control over their own menu and the ability to accept or reject incoming orders.
- Automate the handoff from a validated order to a delivery rider, and let the rider update status as the order moves.
- Give the admin a single view of platform revenue, users, payments, and customer feedback so problems can be caught and acted on.
- Keep the codebase organized and maintainable using an MVC architecture, so new features can be added later without reworking the whole system.

Scope of the Project

The current version of KhabarLagbe covers account registration and login for all four roles; restaurant menu management; browsing and ordering food with a choice of pickup or delivery; order validation and rider assignment by the restaurant; delivery status updates and in-order messaging by the rider; order tracking, tipping, and post-order rating by the customer; and platform-wide user management, revenue reporting, payment tracking, and feedback moderation by the admin.

It does not yet include a live payment gateway (payments are recorded with a method and status rather than processed online), real-time GPS map tracking of riders, or push notifications — these are noted under Future Scope below.

System Architecture

KhabarLagbe follows the MVC (Model-View-Controller) pattern in plain PHP. A single front controller, `index.php`, receives every request from the browser and routes it to the matching controller based on the logged-in user's role and the requested action. Each controller pulls data through its corresponding model, which is the only

layer that talks to the MySQL database, and then hands that data to a view file to render the HTML page that is sent back to the browser. A set of small AJAX endpoint scripts handle actions that need to happen without a full page reload, such as searching food, assigning a rider, sending a message, or updating delivery status.

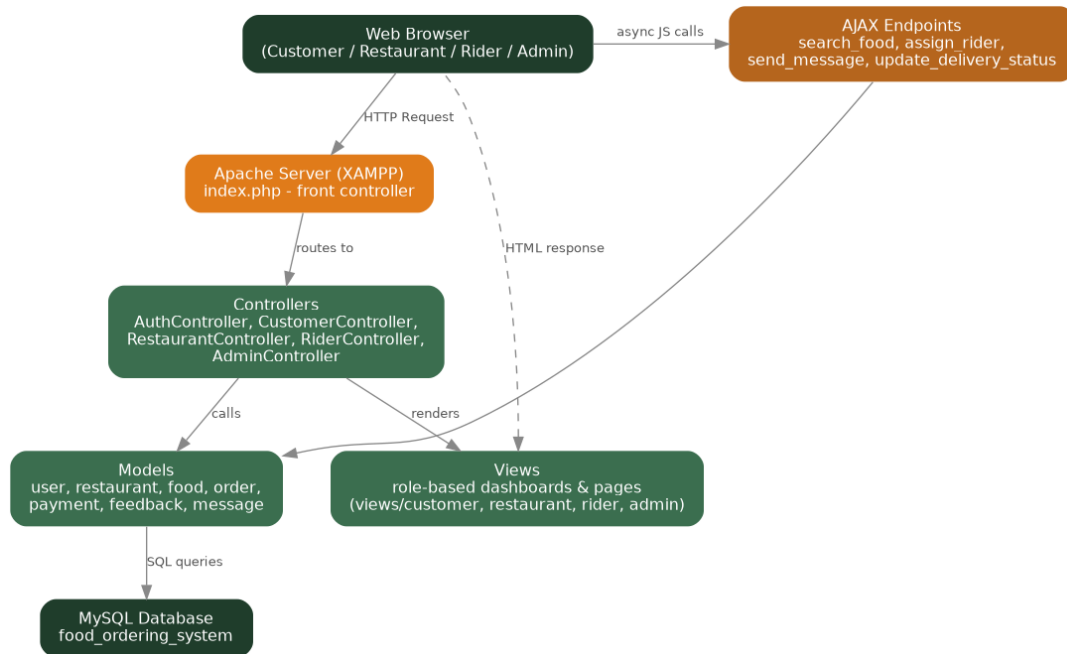


Fig: System architecture — request flow through XAMPP (Apache), the PHP MVC layers, and MySQL

Tools & Technologies Used

- Server Environment: XAMPP (Apache + MySQL + PHP)
- Backend Language: PHP, using an MVC (Model-View-Controller) architecture
- Database: MySQL (database name: food_ordering_system)
- Frontend: HTML, CSS, JavaScript, AJAX (for live search, messaging, and status updates)
- Architecture folders: config, models, controllers, views, ajax, includes, assets

Database Overview

The system's MySQL database is organized around the following core tables:

- users – stores all accounts with a role field (admin, restaurant, customer, rider) and status (active/suspended)
- restaurants – restaurant profile linked to a restaurant-role user
- foods – menu items belonging to a restaurant, with price, category, and availability status
- orders – links customer, restaurant, and rider; tracks delivery type (pickup/delivery), total amount, tip, and status (pending → accepted → preparing → assigned → on the way → delivered/picked up, or rejected/cancelled)
- order_items – individual food items and quantities within an order
- payments – payment method (cash, card, mobile banking) and status (pending, paid, refunded)
- feedback – customer ratings and comments per order, with an admin_action field for follow-up
- delivery_status_log – history of status updates made by the rider for each order
- messages – in-order messaging between customers and riders/restaurants

ER Diagram

The diagram below shows how the core tables relate to one another. A user (role: customer) places many orders; each order belongs to one restaurant and, once assigned, to one rider. Each order contains one or more order_items, each pointing to a food item. Each order has one payment record and, after completion, may receive one feedback entry. Riders write entries to delivery_status_log as they update an order's status, and customers and riders exchange messages tied to a specific order.

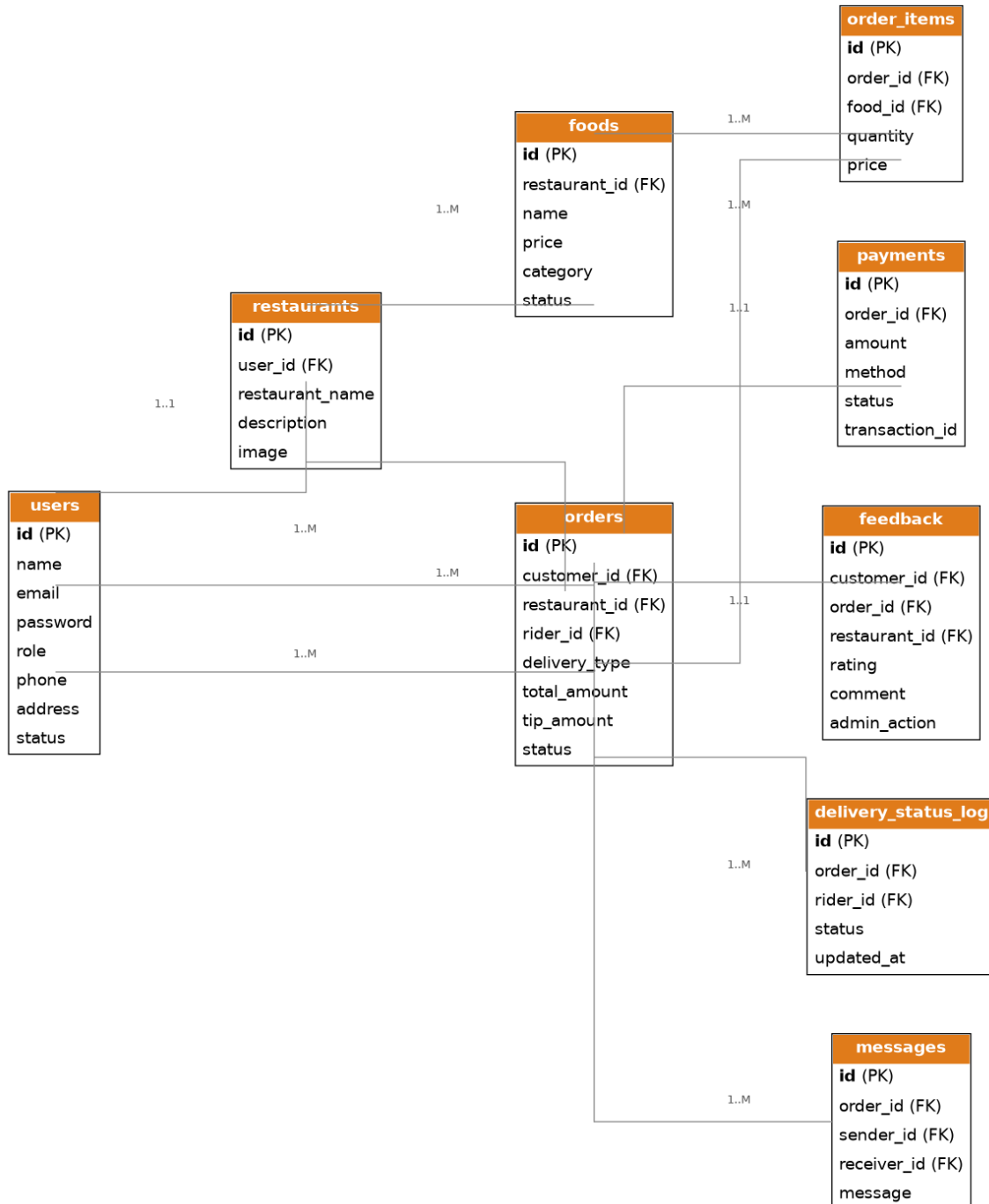


Fig: Entity-Relationship diagram of the KhabarLagbe database (food_ordering_system)

Order Lifecycle / Workflow

Every order moves through a defined sequence of statuses, tracked in the orders table and logged per update in delivery_status_log. This keeps the customer, restaurant, and rider in sync on where an order currently stands.

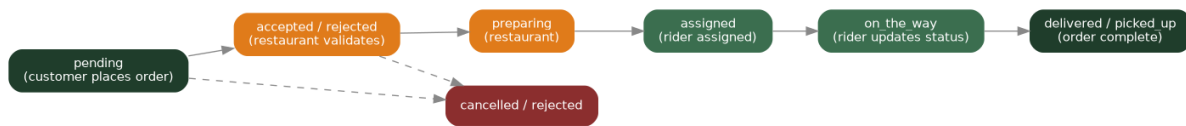


Fig: Order status workflow, from placement to delivery/pickup, with cancellation/rejection paths

Types of Users

1. Admin 2. Restaurant 3. Rider 4. Customer

Common Features (Available to All Users)

Authentication

1. Login to the system
2. User registration (signup)
3. Logout from the system

Dashboard

1. Access a personalized, role-specific dashboard after login, showing key stats at a glance

Specific Features by User Type

Features of Admin

1. View platform-wide stats: total revenue, completed orders, total users, feedback received
2. Revenue Report – view overall platform revenue
3. Action on Feedback – review customer feedback and take action (warn / suspend / resolve)
4. Payment Management – monitor and manage payment records
5. Manage Users – full CRUD and search over all user accounts
6. All Orders – view and search every order placed on the platform

Features of Restaurant

1. View dashboard stats: menu items, total orders, pending validation
2. Add / Edit / Delete Food – manage the restaurant's own menu items
3. Order Validation – accept or reject incoming customer orders
4. Assign Delivery Rider – assign an available rider to a validated order

Features of Rider

1. View dashboard stats: total assigned, active deliveries, completed deliveries
2. Assigned Deliveries List – view all deliveries currently assigned
3. Update delivery status (e.g. picked up, on the way, delivered)
4. Message Customers – communicate directly with customers about their delivery

Features of Customer

1. View dashboard stats: total orders, active orders
2. Browse & Order Food – search restaurants/menu items and place an order
3. My Orders – track order status, choose pickup or delivery, and add a tip
4. Rate & Review – leave feedback and a rating after an order is completed

Use Case Diagrams

The use cases are grouped below by actor. All four roles share the common Login / Signup / Logout / Dashboard use cases; each role additionally has the specific use cases shown in the feature list above.

Common (All Roles)

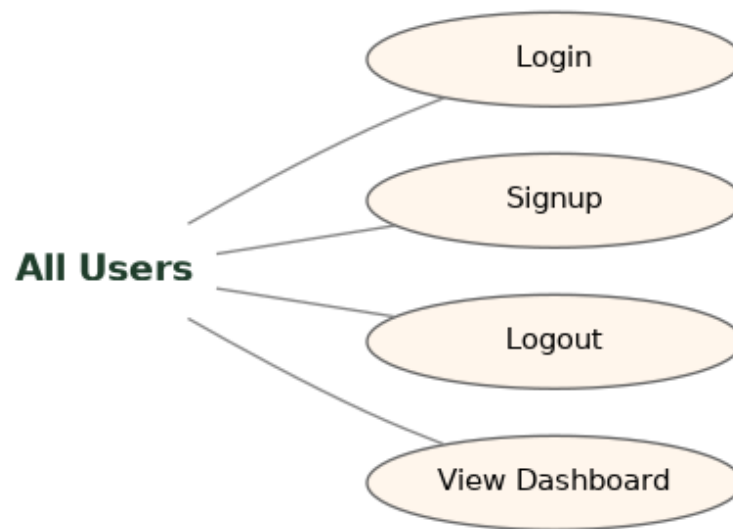


Fig: Use cases shared by every role

Customer

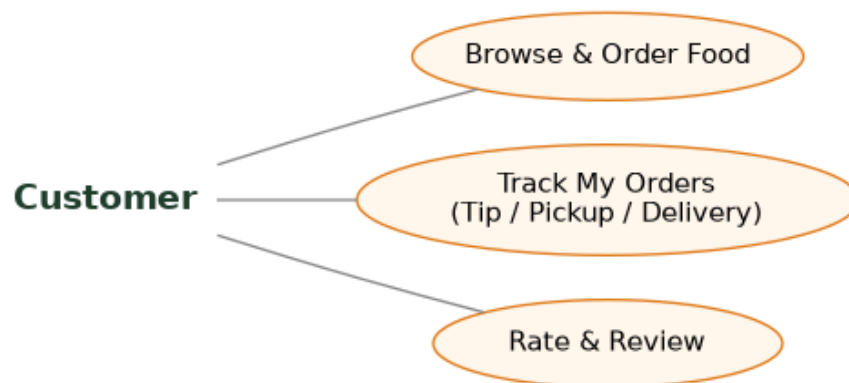


Fig: Customer use cases

Restaurant Owner

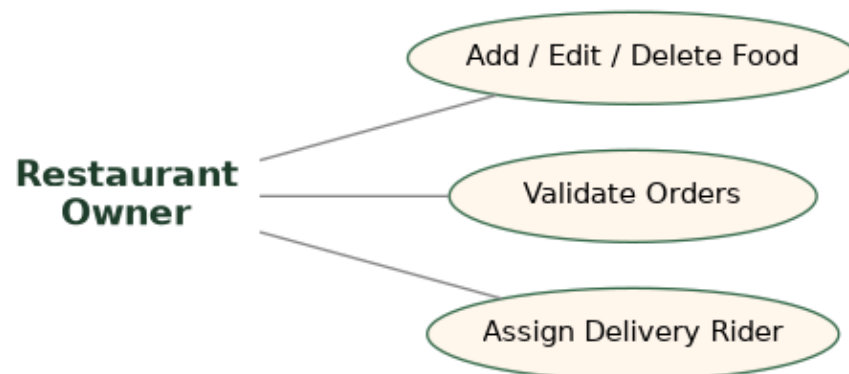


Fig: Restaurant owner use cases

Delivery Rider

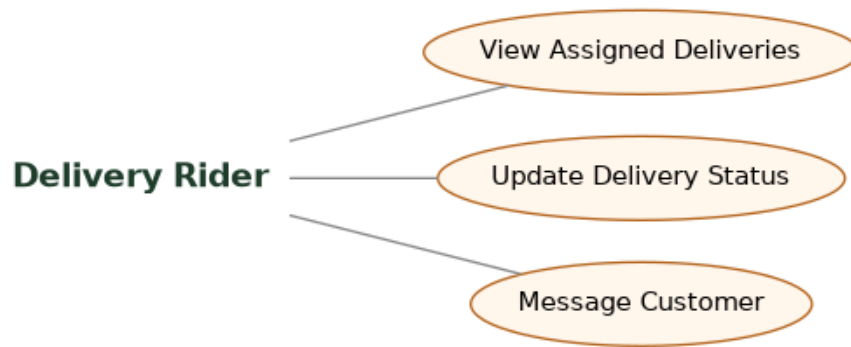


Fig: Delivery rider use cases

Admin

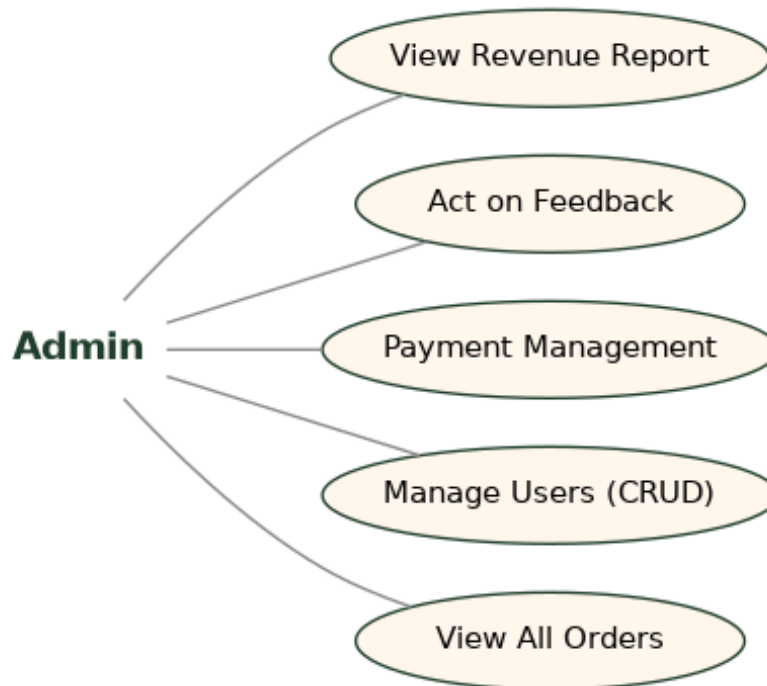


Fig: Admin use cases

System Dashboards (UI Screenshots)

The following screenshots show the role-based dashboards implemented in the current version of the system, running locally on XAMPP.

Customer Dashboard

After logging in, the customer lands on a dashboard summarizing their Total Orders and Active Orders. From here they can jump straight into Browse & Order Food to place a new order, into My Orders to track existing orders (including choosing pickup or delivery and adding a tip), or into Rate & Review to leave feedback once an order is complete.

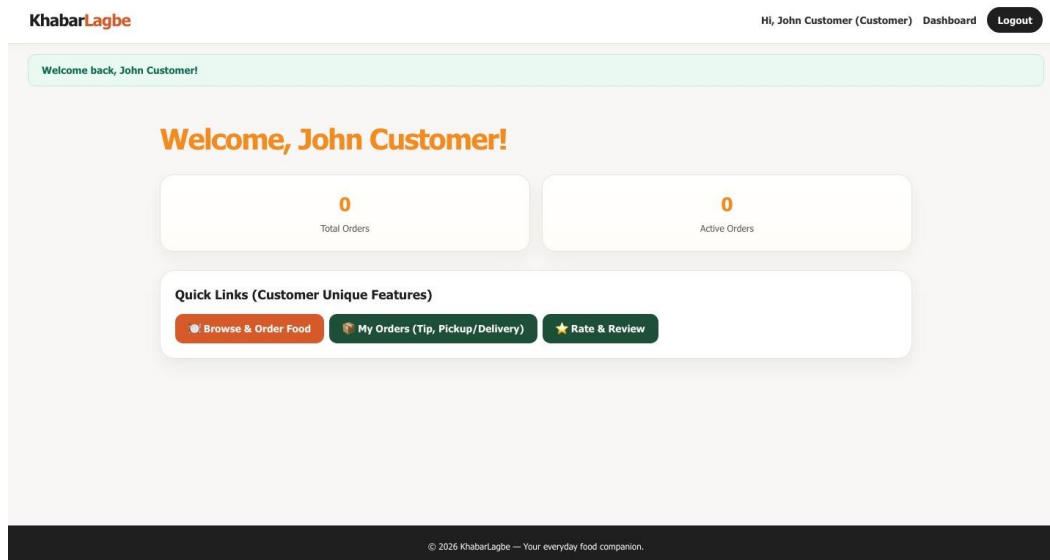


Fig: Customer dashboard – order stats and quick links to Browse & Order Food, My Orders, and Rate & Review

Restaurant Dashboard

The restaurant owner's dashboard (shown here for “Pizza Palace”) shows Menu Items, Total Orders, and orders still Pending Validation at a glance. Quick links let the owner add, edit, or delete food items, validate (accept/reject) incoming orders, and assign a delivery rider once an order is ready.

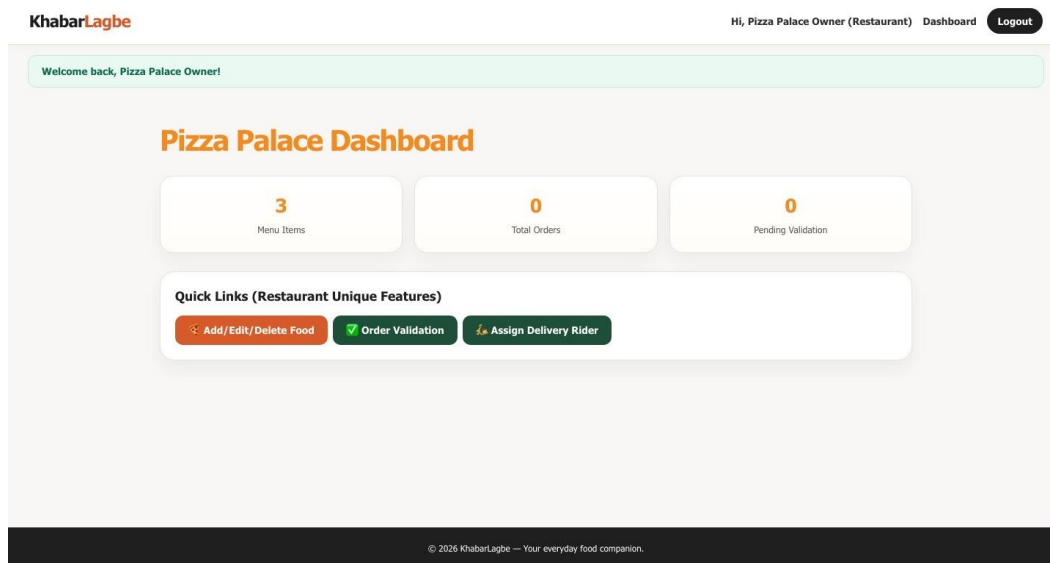


Fig: Restaurant dashboard – menu/order stats and quick links to manage food, validate orders, and assign riders

Rider Dashboard

The rider's dashboard tracks Total Assigned, Active Deliveries, and Completed deliveries. From here the rider opens the Assigned Deliveries List to see and update the status of each delivery, or uses Message Customers to communicate directly about a drop-off.

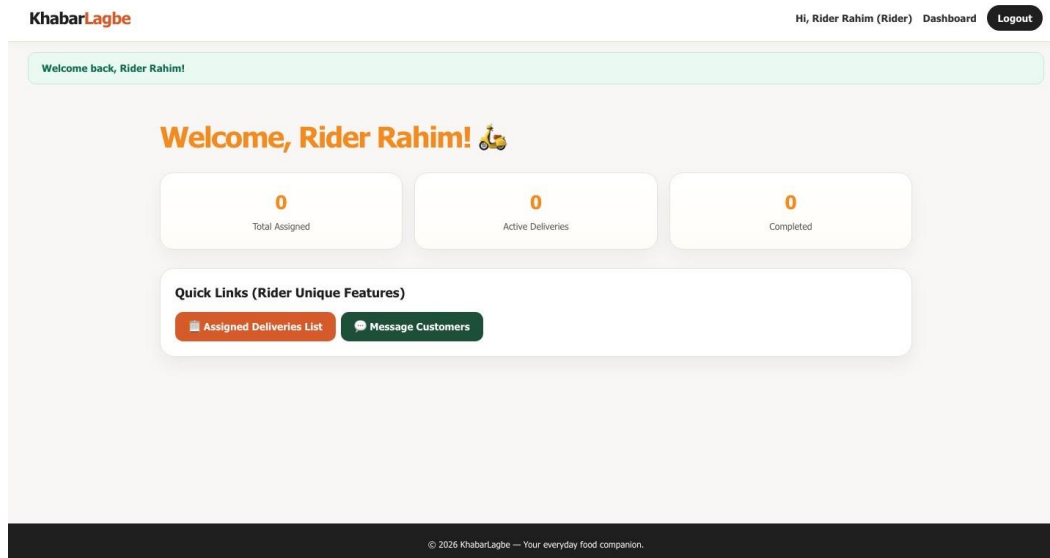


Fig: Rider dashboard – delivery stats and quick links to assigned deliveries and customer messaging

Admin Dashboard

The admin dashboard gives a platform-wide view: Total Revenue, Completed Orders, Total Users, and Feedback Received. Below the stats, the admin can open the Revenue Report, act on customer Feedback, manage Payments, manage all Users (full CRUD and search), and view All Orders across every restaurant.

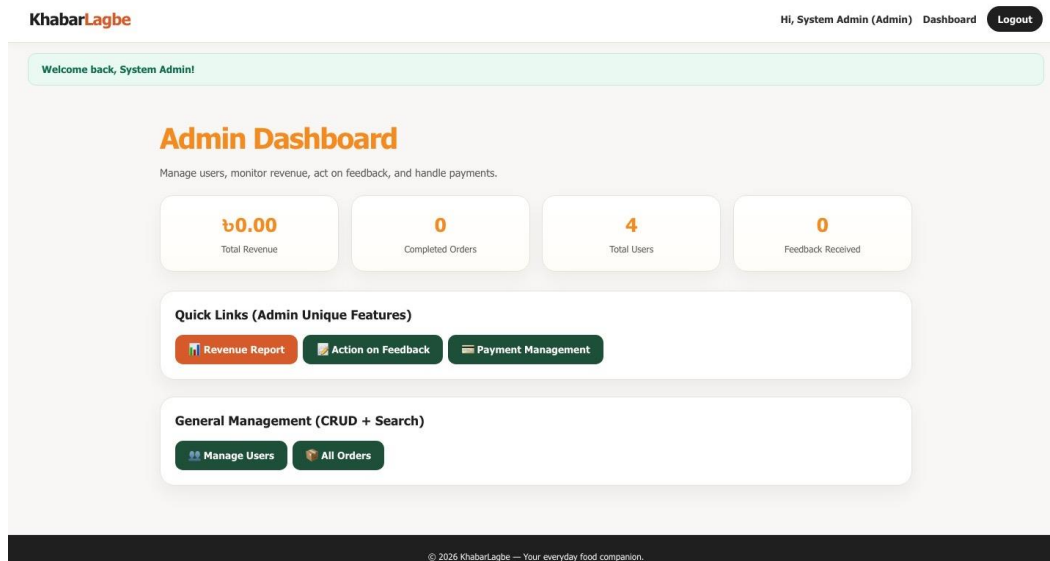


Fig: Admin dashboard – platform-wide stats and quick links to revenue, feedback, payments, users, and orders

Limitations & Future Scope

The current version is a working local prototype (run through XAMPP) rather than a deployed production system, so a few areas are natural next steps:

- Online payment gateway – integrate a real payment processor (e.g. card or mobile banking API) instead of manually recorded payment status.
- Real-time rider tracking – add live GPS location so customers can see the rider's position on a map.
- Push/SMS notifications – notify customers and riders instantly on order status changes, instead of requiring a dashboard refresh.

- Ratings-based rider/restaurant ranking – use accumulated feedback to rank restaurants and riders and surface top performers.
- Mobile app – package the customer and rider experiences as native or hybrid mobile apps.
- Automated testing and deployment – add a test suite and move from a local XAMPP setup to a hosted production server.

Expected Outcome

- A fully functional, role-based food ordering platform running on PHP + MySQL via XAMPP
- Smooth order flow from customer placement → restaurant validation → rider assignment → delivery
- Centralized management of users, payments, and feedback for the Admin
- Improved communication between customers and riders through in-app messaging
- A foundation that can be extended with online payment gateways, live GPS tracking, and a mobile app in the future

Conclusion

KhabarLagbe demonstrates how a single web application, built with a clean MVC structure in PHP and MySQL, can bring together customers, restaurants, riders, and an administrator into one coordinated food ordering and delivery workflow. By replacing manual, phone-based coordination with structured dashboards, order tracking, and role-based features, the system reduces errors, improves transparency, and lays a solid foundation for future enhancements such as online payments and real-time delivery tracking.